



02456 – Week 7

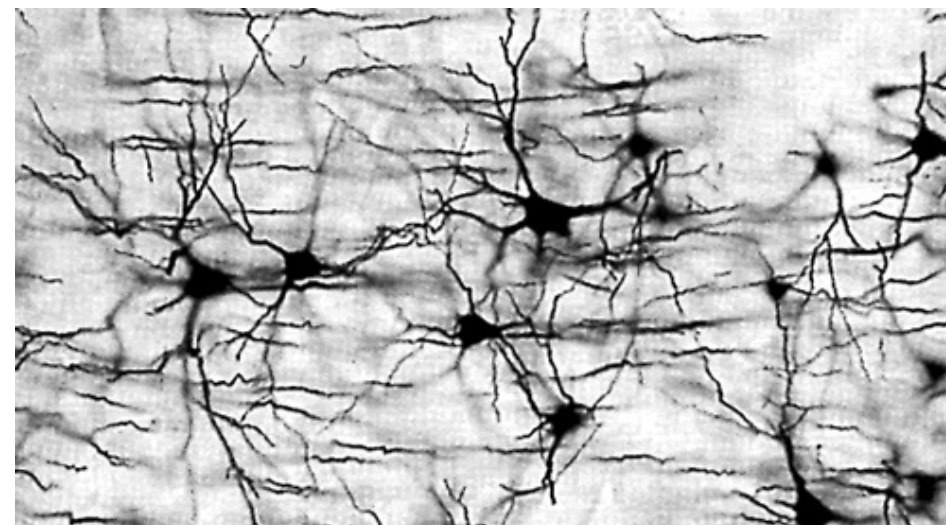
Unsupervised

deep learning

Jes Frellsen

Technical University of Denmark

20 October 2025



Menu of the day

- Week 1: Neural nets
- Week 2: Learning
- Week 3: Tricks of The Trade
- Week 4: CNNs
- Week 5: RNNs
- Week 6: Transformers
- **Week 7: Unsupervised**
- Week 8: Mini-project

Lecture

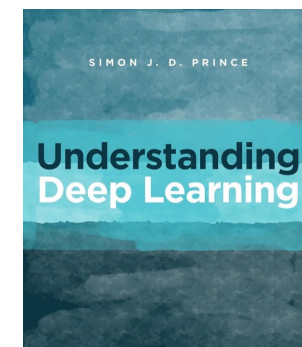
- Auto-encoder (AE)
- Variational autoencoder (VAE)
- Generative adversarial network (GAN)

Reading

- Chapters 14, 15 & 17

Exercises

- **Notebooks 7.1-7.3:**
 - AE, VAE, GANs
- **Problems:** 15.2, 17.4



Announcements

- **Project list** is available on

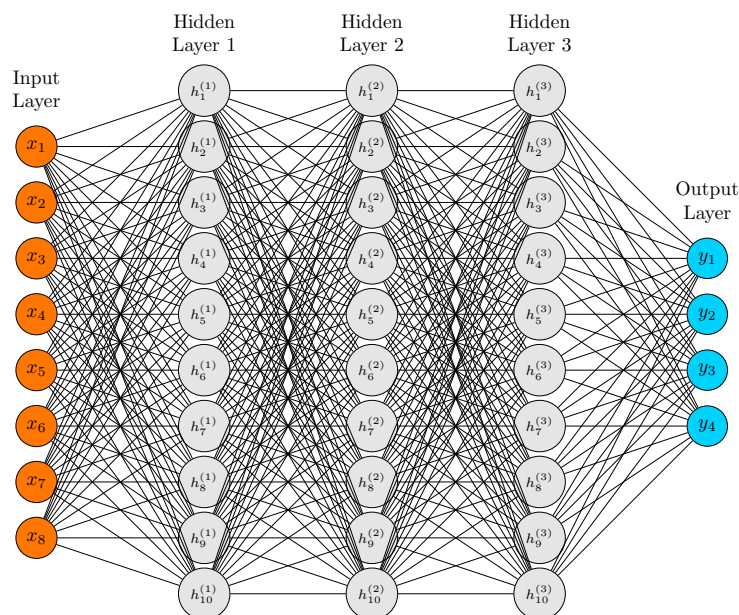
DTU Learn → Content → Project (weeks 9-13)

Project selection deadline: Monday, Week 8, 23:59

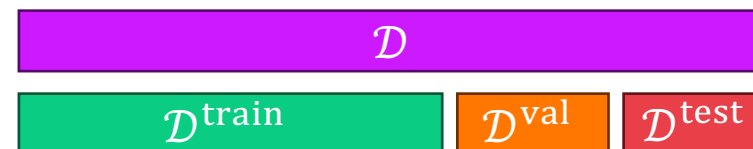
28 projects different projects

- No lecture in week 8 (mini-project)
- Project runs weeks 9-13

Recap: Supervised deep learning



- A **loss function** $L(\phi)$ captures the **mismatch** of $f_\phi(\mathbf{x}_i)$ and y_i : $\hat{\phi} = \operatorname{argmin}_\phi L(\phi)$
- Use the **NLL** $L(\phi) = -\sum_i \log p(y_i | f_\phi(\mathbf{x}_i))$
- Use **gradient descent** to find $\hat{\phi}$
- Split **data set** in three



We can write the **output of layer ℓ** as

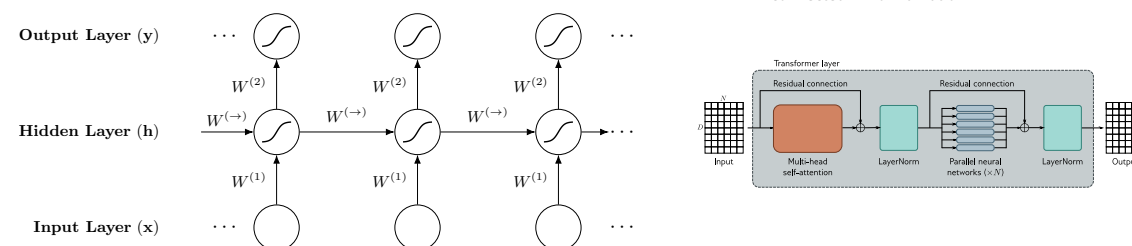
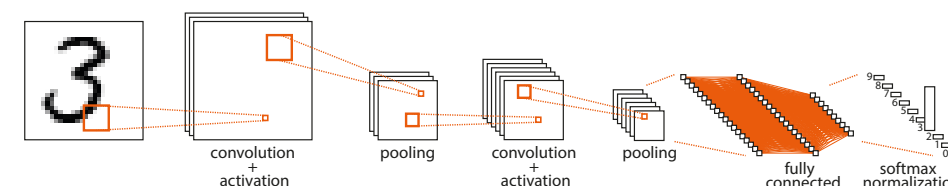
$$f^{(\ell)}(\mathbf{h}) = \sigma(W^{(\ell)}\mathbf{h} + \mathbf{b}^{(\ell)}),$$

where σ is a non-linear **activation function**.

The **joint function** is then

$$f_\phi(\mathbf{x}) = \mathbf{y} = f^{(4)}\left(f^{(3)}\left(f^{(2)}\left(f^{(1)}(\mathbf{x})\right)\right)\right)$$

- Special architectures for images and sequences



Unsupervised deep learning

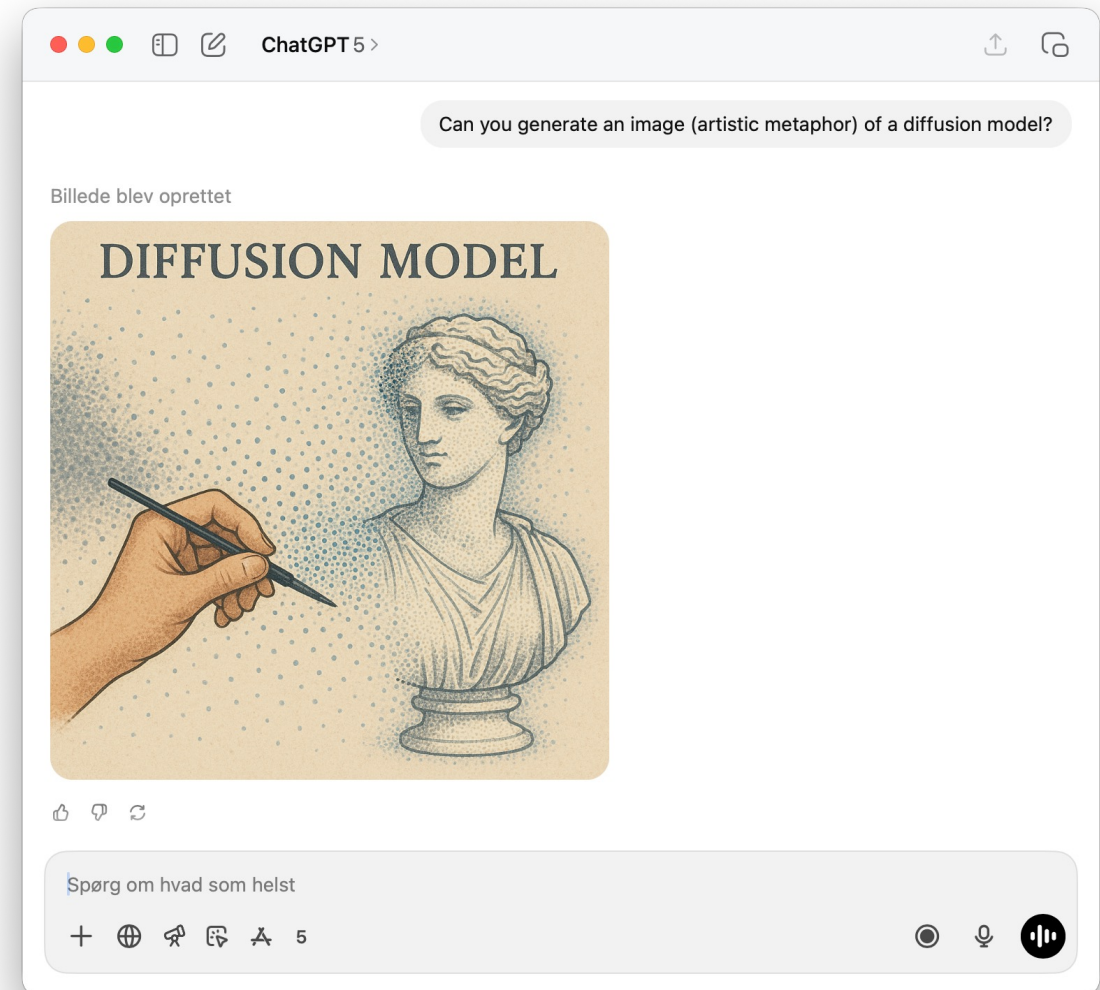
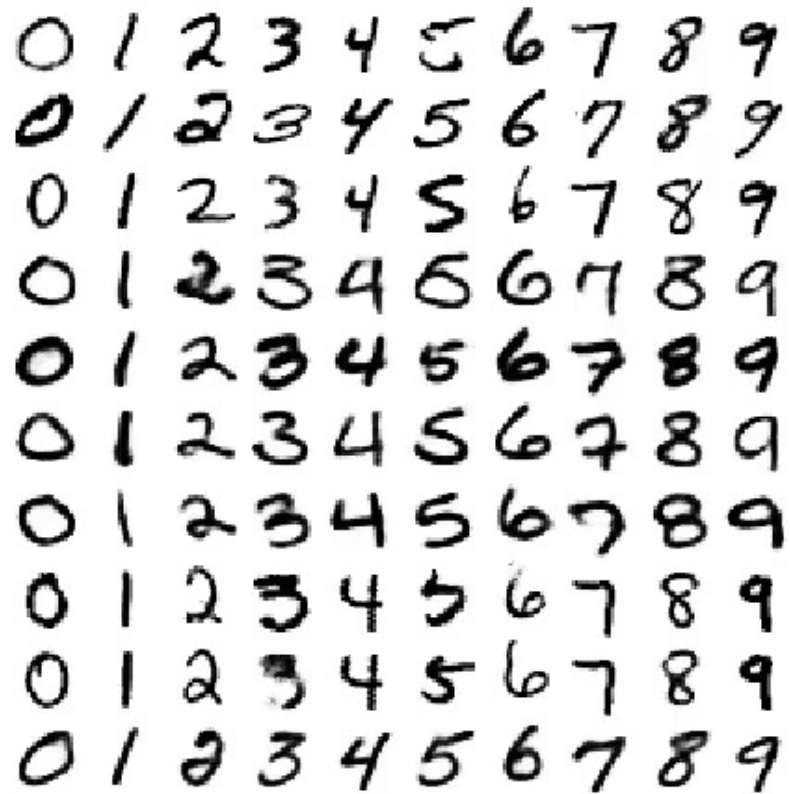


- Deep learning in the **early 2010s**:
 - Mostly about supervised learning
 - Requires a lot of labelled data: expensive to collect
 - The internet has loads of unlabelled data (images, text, etc.)
- Deep learning in the **2020s**:
 - Unsupervised, more human-like
 - Humans can easily generalise from few labelled examples

“We expect unsupervised learning to become far more important in the longer term. Human and animal learning is largely unsupervised: we discover the structure of the world by observing it, not by being told the name of every object.”

---LeCun, Bengio, Hinton, Nature 2015

Generative models for complex data

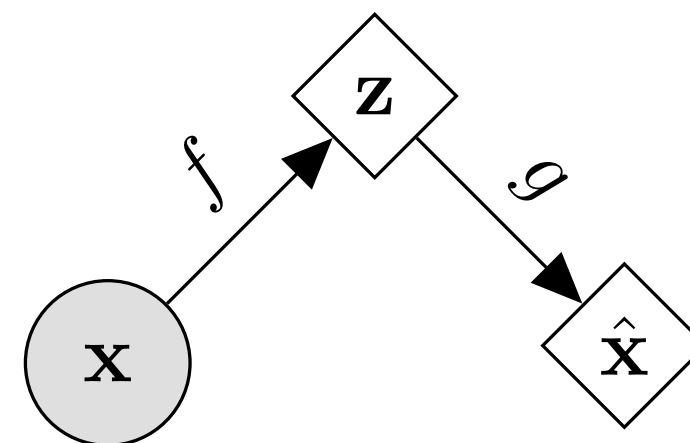


Unsupervised learning

- **Data is just $\mathbf{x}$** , not input-output pairs $(\mathbf{x}, y)$
- **Goal:**
 - Build a model $p_{\theta}(\mathbf{x})$ of the data generating distribution $p_{\text{data}}(\mathbf{x})$
 - Usually learned through maximum likelihood for training data $\{\mathbf{x}_i\}_i$
$$\hat{\theta} = \operatorname{argmax}_{\theta} \ell(\theta) \quad \text{where} \quad \ell(\theta) = \sum_i \log p_{\theta}(\mathbf{x}_i)$$
- **Side goal:**
 - Learn a (low dimensional) **representation** $f: \mathbf{x} \mapsto \mathbf{z}$
 - E.g., with a **latent variable** model $p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z}$
- Comparisons to supervised learning, where we learn $p_{\theta}(y|\mathbf{x})$:

Approaches to unsupervised learning

- Many unsupervised learning approaches use **latent variables $\mathbf{z}$**
 - **Discrete $\mathbf{z}$** : k -means, mixture models, ...
 - **Continuous $\mathbf{z}$** : PCA, ICA, NMF, state space models, ...
- These models can be thought of as **autoencoders**:
 - **Encoder** function: $f: \mathbf{x} \mapsto \mathbf{z}$
 - **Decoder** function: $g: \mathbf{z} \mapsto \mathbf{x}$
 - **Reconstruction**: $\hat{\mathbf{x}} = g(f(\mathbf{x}))$
 - We want $\hat{\mathbf{x}} \approx \mathbf{x}$

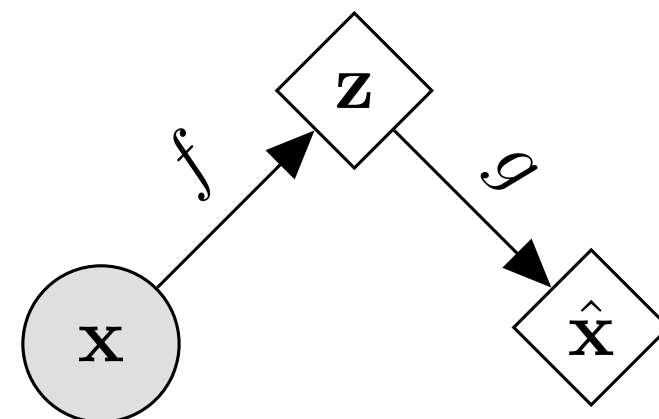


PCA as an autoencoder

- Assume **linear encoder and decoder**:

$$f(\mathbf{x}) = W_e \mathbf{x}$$

$$g(\mathbf{z}) = W_d \mathbf{z}$$



- PCA solution minimises the criterion

$$L = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\|\mathbf{x} - g(f(\mathbf{x}))\|^2] = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\|\mathbf{x} - \hat{\mathbf{x}}\|^2]$$

- Recovers same subspace as PCA, but basis is not orthonormal

Is there an issue?
Is this problem easy or hard?

If $\dim(\mathbf{x}) = \dim(\mathbf{z})$ the identity mapping $g(f(\mathbf{x})) = \mathbf{x}$ is easy with

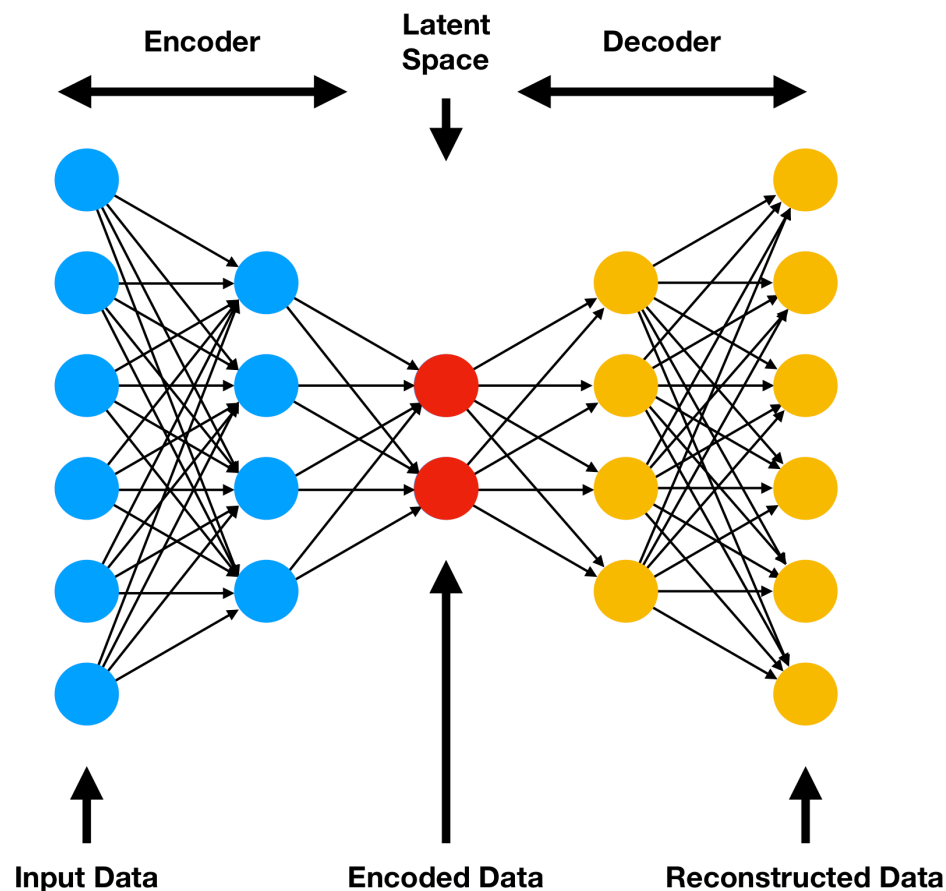
$$W_d = W_e^{-1}$$

How to make the autoencoding problem harder?

Autoencoders

- Regularisation avoids learning the identity function:
 - **Bottleneck autoencoder**: limit the dimensionality of $\mathbf{z}$
(Bourlard and Kamp, 1988, Oja, 1991)
 - **Sparse autoencoder**: penalize activations of $\mathbf{z}$
(Ranzato et al., 2006, Le et al., 2011)
 - **Denoising autoencoder**: inject noise to input $\mathbf{x}$
(Vincent et al., 2008)
 - **Contractive autoencoder**: penalize Jacobian of $f(\cdot)$
(Rifai et al., 2011)
 - ...

(Bottleneck) autoencoder

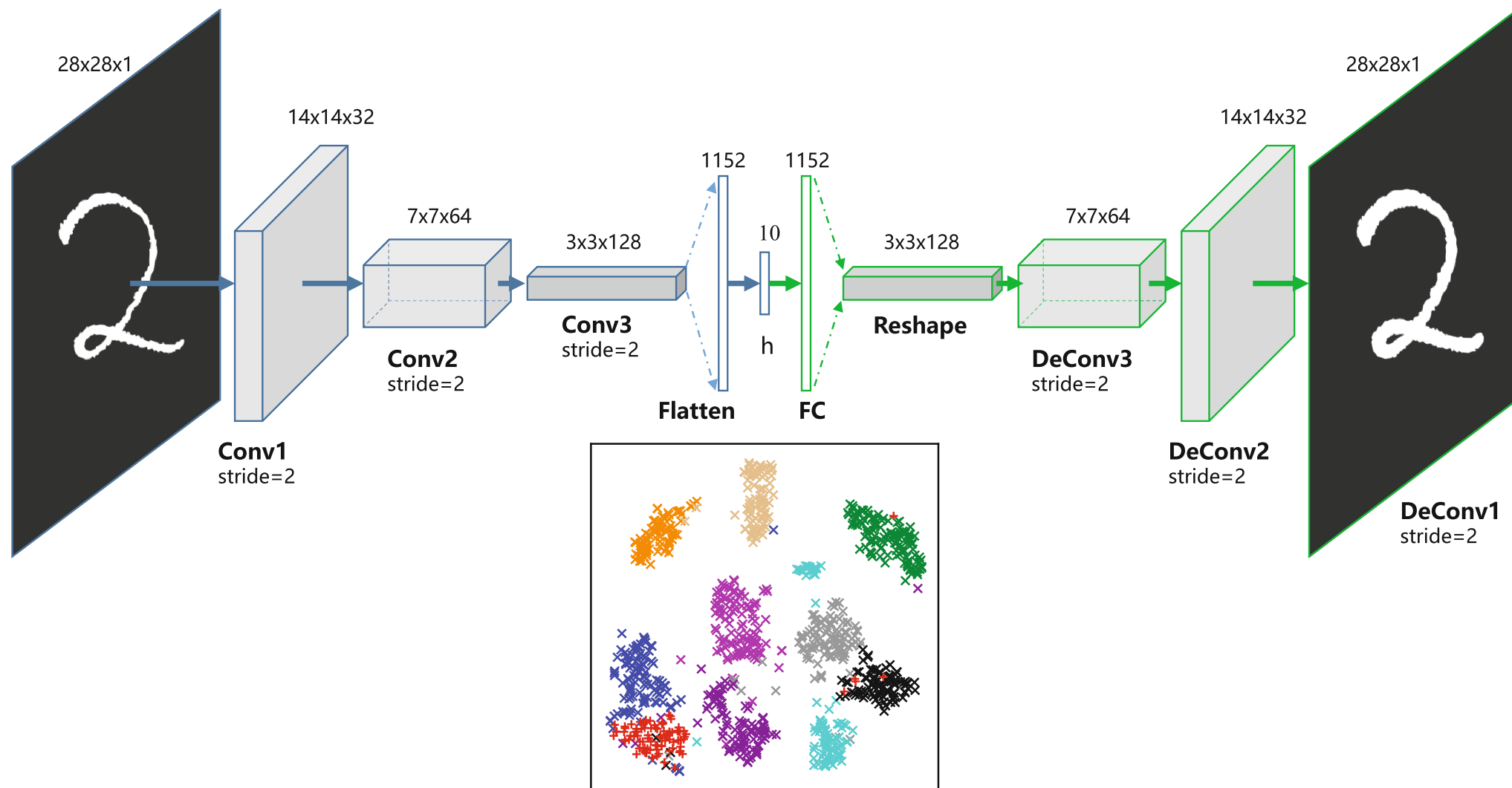


Loss is

$$L(\phi_e, \phi_d) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} \left[\left\| \mathbf{x} - g_{\phi_d} \left(f_{\phi_e}(\mathbf{x}) \right) \right\|^2 \right]$$

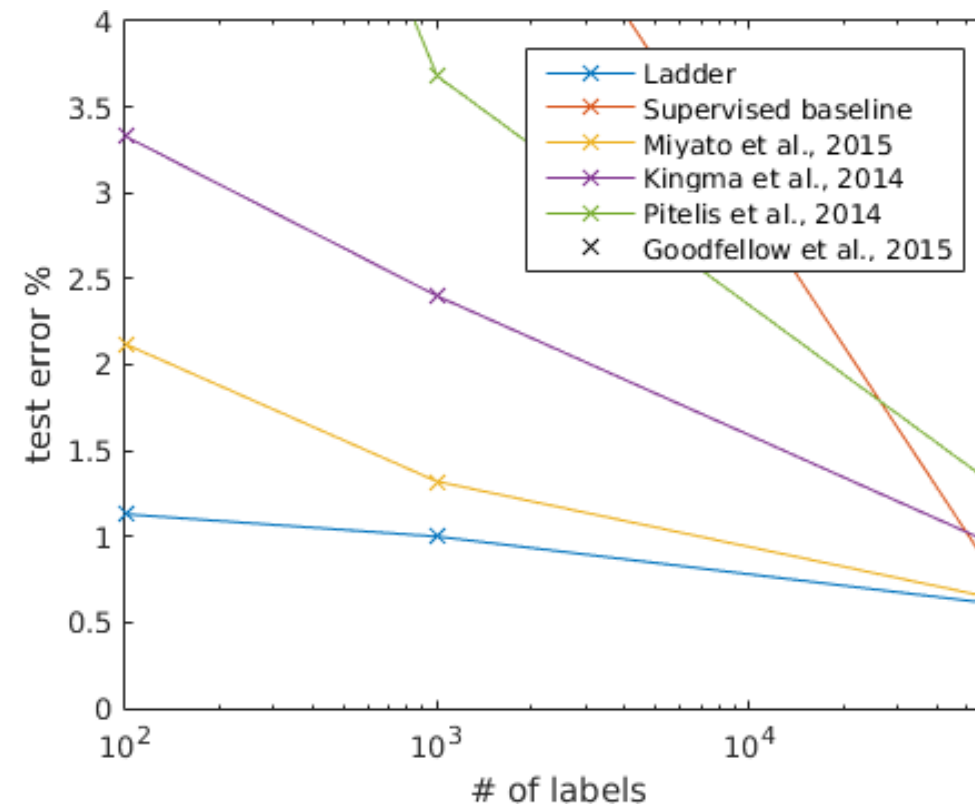
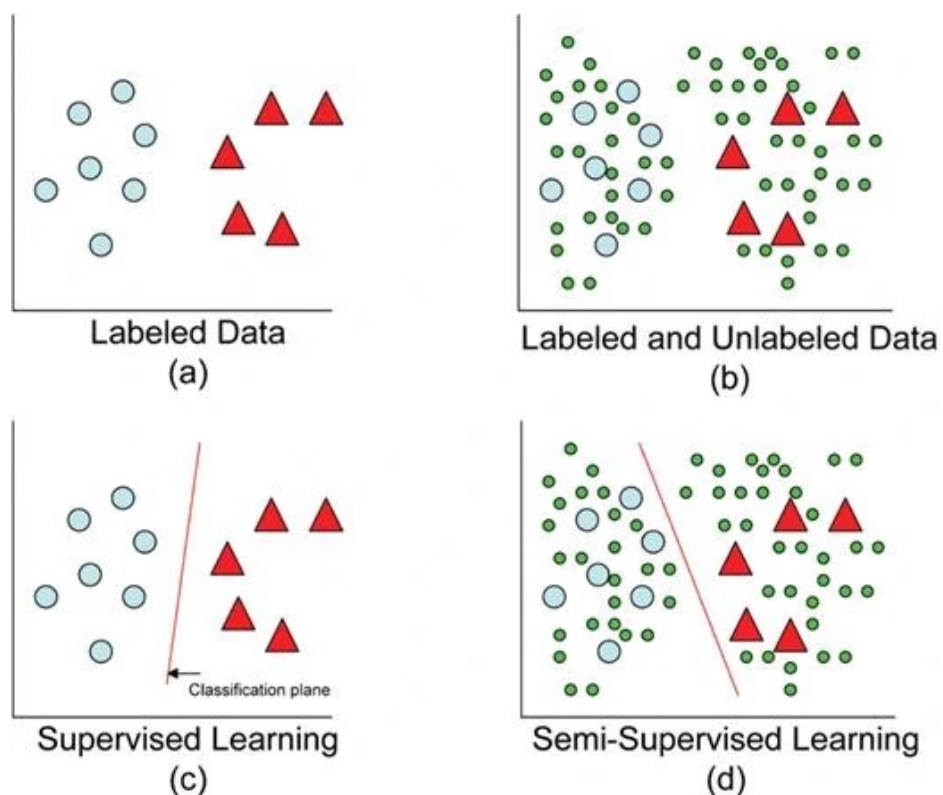
where f and g are neural nets

Convolutional autoencoders



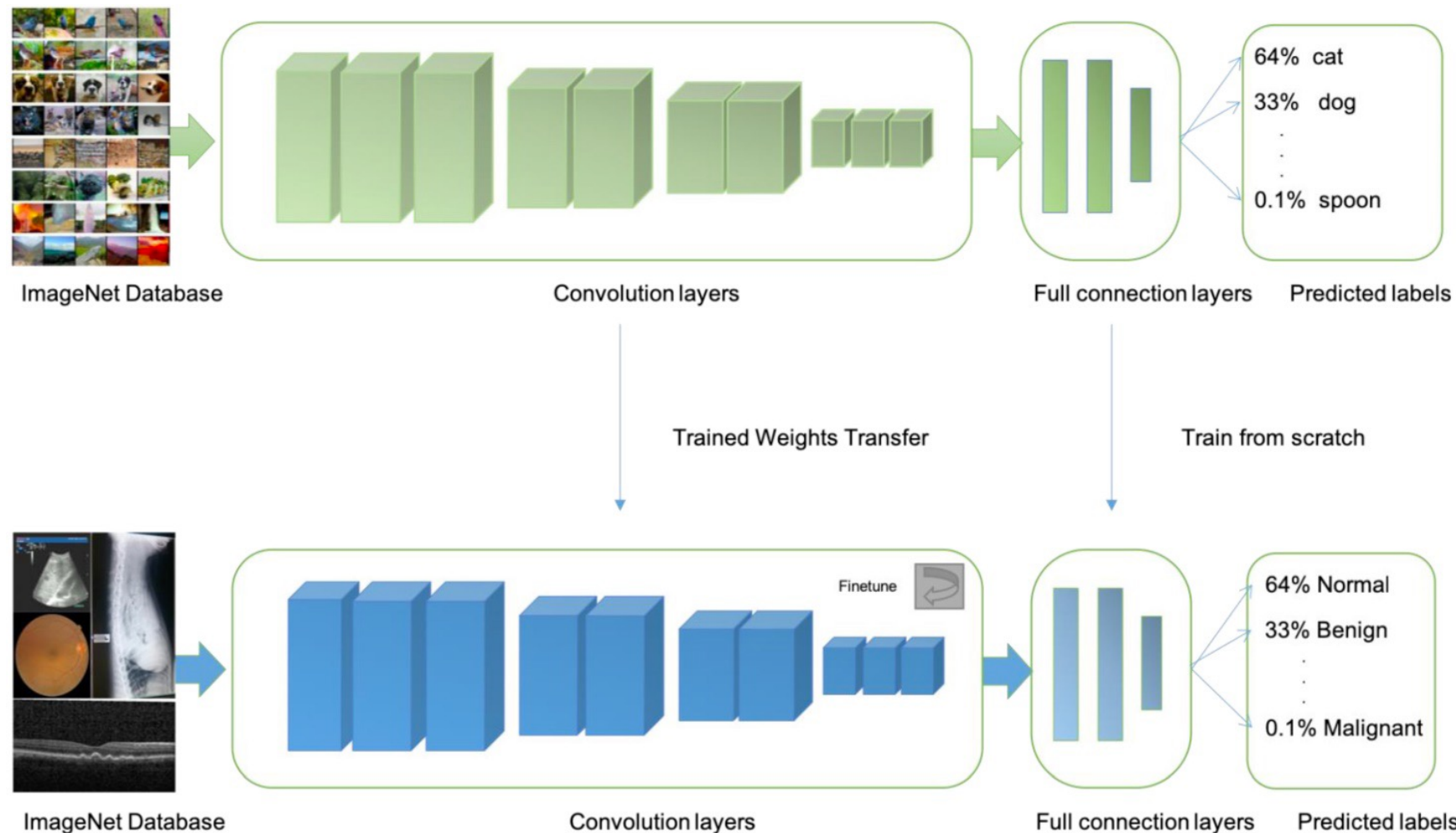
Semi-supervised learning

- We can use the representation for semi-supervised learning



Side note:

Transfer learning with a supervised model



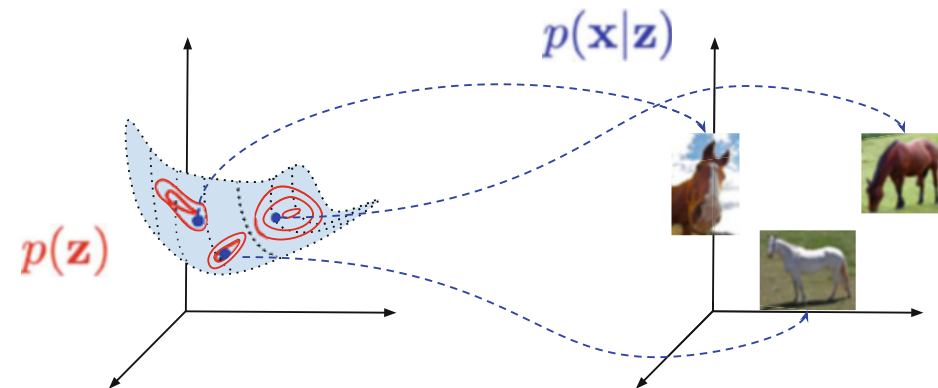
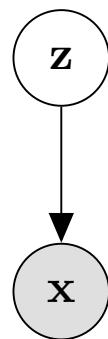
Limitation of auto encoder?

Latent variable models

- Generative process**

$$\mathbf{z} \sim p(\mathbf{z})$$

$$\mathbf{x} \sim p(\mathbf{x}|\mathbf{z})$$



- Manifold hypothesis:** $\mathbf{z} \in \mathbb{R}^M$ and $\mathbf{x} \in \mathbb{R}^D$ where $M < D$

- We only observe $\mathbf{x}$, and the **marginal density of $\mathbf{x}$** is

$$p(\mathbf{x}) = \int p(\mathbf{x}|\mathbf{z})p(\mathbf{z}) d\mathbf{z}$$

- Posterior distribution (probabilistic encoding): $p(\mathbf{z}|\mathbf{x}) = \frac{p(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p(\mathbf{x})}$

- Learning by **maximum likelihood**:

$$\hat{\theta} = \operatorname{argmax}_{\theta} \ell(\theta) \quad \text{where} \quad \ell(\theta) = \sum_i \log p_{\theta}(\mathbf{x}_i)$$

Difficult
computation if
the
distributions
are not simple

Deep latent variable models (DLVMs)

Assume $\mathbf{x}, \mathbf{z}$ are random variables driven by the model:

$$\mathbf{z} \sim p(\mathbf{z})$$

(prior)

$$\mathbf{x} \sim p_{\theta}(\mathbf{x}|\mathbf{z}) = \Phi(\mathbf{x}|f_{\theta}(\mathbf{z}))$$

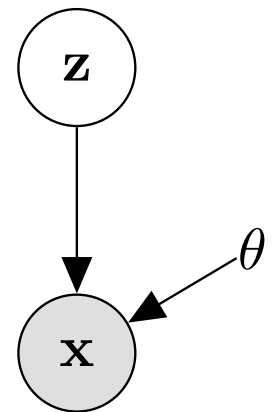
(observation model)

where

- $\mathbf{z} \in \mathcal{Z}^M$ is the **continuous latent** variable,
- $\mathbf{x} \in \mathcal{X}$ is the **observed** variable,
- the function $f_{\theta}: \mathcal{Z}^M \rightarrow H$ (**deep**) **neural network** called the **decoder**,
- $\{\Phi(\cdot | \eta)\}_{\eta \in H}$ is a parametric family called the **output density**.

Usually a standard isotropic Gaussian $\mathcal{N}(\mathbf{0}_M, I_M)$

The output density is usually **very simple**: unimodal and fully factorised, e.g., multivariate Gaussian or products of binomials or multinomials.



Illustrative example of a DLVM

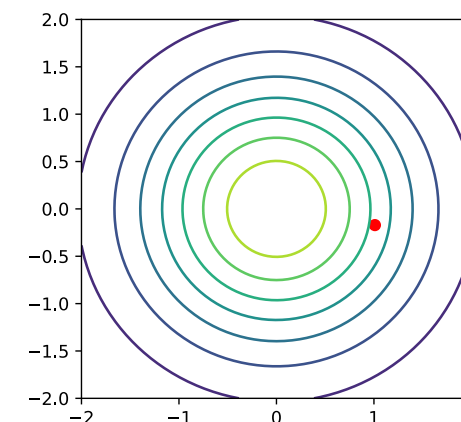
Training data $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ binary MNIST



Generation

$$\mathbf{z} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\right)$$

$$\mathbf{z} = (1.0097, -0.1711)$$



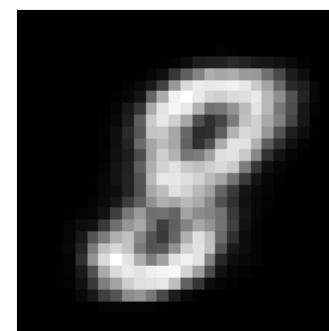
Generative model for $\mathbf{z} \in \mathbb{R}^2$ and $\mathbf{x} \in \{0, 1\}^{28 \times 28}$

$$\begin{cases} \mathbf{z} \sim \mathcal{N}\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\right) \\ x^{j,k} \sim \text{Bernoulli}(p = f^{j,k}(\mathbf{z})) \end{cases}$$

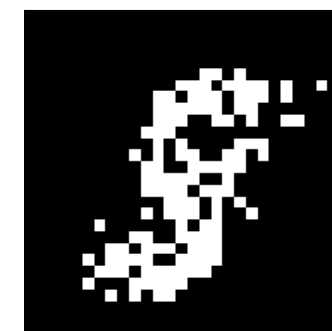
Decoder network

$$f(\mathbf{z}) = \text{Sigmoid}(\mathbf{V} \tanh(\mathbf{W}\mathbf{z} + \mathbf{b}) + \boldsymbol{\beta})$$

$f(\mathbf{z})$



$x^{j,k} \sim \text{Bern}(f^{j,k}(\mathbf{z}))$



Learning objective

- We want to **learn the parameters θ of the decoder**
- Given observations $\{\mathbf{x}_1, \dots, \mathbf{x}_N\}$, the **log-likelihood** objective is

$$\ell(\theta) = \sum_{n=1}^N \log p(\mathbf{x}_n) \quad \text{where} \quad p(\mathbf{x}_n) = \int p_{\theta}(\mathbf{x}_n|\mathbf{z})p(\mathbf{z}) \, d\mathbf{z}$$

- We would like to find the **ML estimate**: $\hat{\theta} = \arg \max_{\theta} \ell(\theta)$.
- However, $p(\mathbf{x}_n)$ is **intractable** for non-linear decoders

Amortized variational inference (VI)

- VI approximatively maximises the log-likelihood, $\ell(\theta)$, by maximising the ELBO

$$\begin{aligned}\mathcal{L}(\theta, \phi) &= \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_{\theta}(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right] \\ &= \underbrace{\mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|\mathbf{z})]}_{\text{reconstruction term}} - \underbrace{\text{KL}[q_{\phi}(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z})]}_{\text{regularisation term}} \leq \ell(\theta)\end{aligned}$$

- We use the **variational posterior**

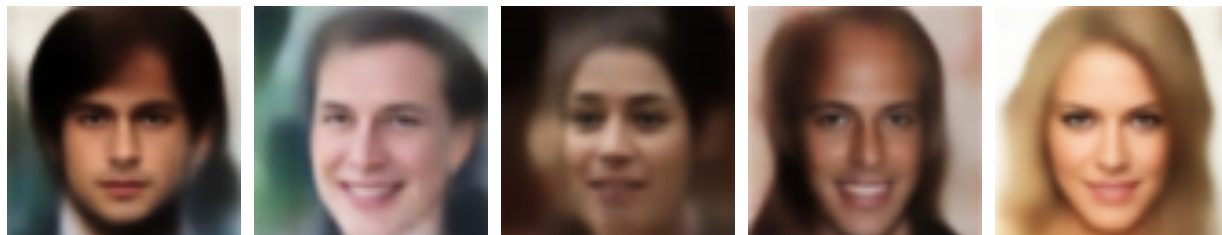
$$q_{\phi}(\mathbf{z}|\mathbf{x}) = \Psi(\mathbf{z}|\mathbf{g}_{\phi}(\mathbf{x}))$$

Closed form for
Gaussians

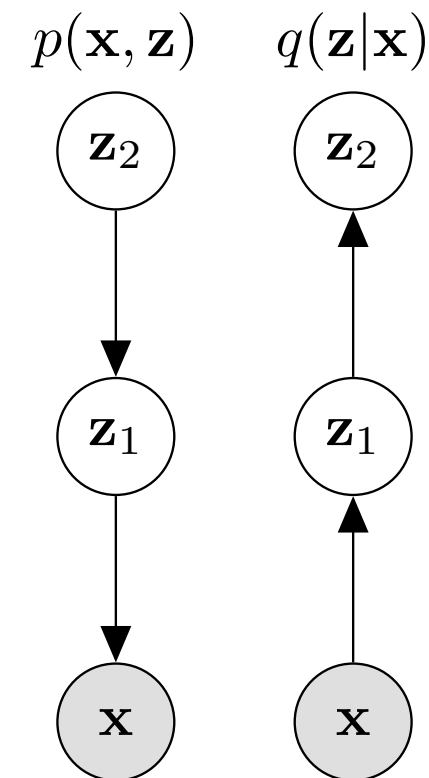
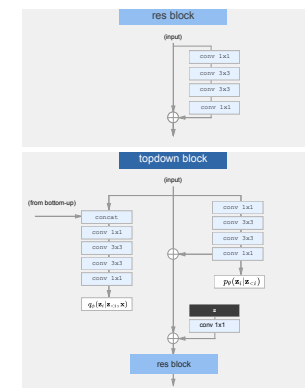
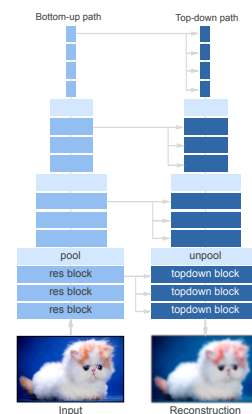
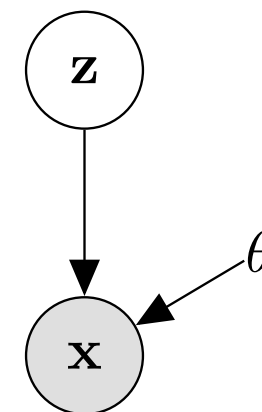
where

- $\{\Psi(\cdot|\kappa)\}_{\kappa \in K}$ is the **variational family** over $\mathcal{Z}^M$ (for $\mathbb{R}^M$ usually Gaussians)
- $\mathbf{g}_{\phi}: \mathcal{X} \rightarrow K$ is a neural net called the **encoder** or **inference network**
- $\mathbf{g}_{\phi}$ transforms a datapoint $\mathbf{x}$ into parameters of the variational distribution
- The variational distribution is a **tractable approximation** of the posterior $q_{\phi}(\mathbf{z}|\mathbf{x}) \approx p_{\theta}(\mathbf{z}|\mathbf{x})$
- Low variance gradients, $\nabla_{\theta, \phi} \mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} [f_{\theta, \phi}(\mathbf{z})]$, calculated using the **reparameterization trick**

Image generation with VAEs



What is the issue with these images?



Top: Larsen et al. Autoencoding beyond pixels using a learned similarity metric. ICML, 2016.

Bottom: Child R. Very deep VAEs generalize autoregressive models and can outperform them on images. arXiv:2011.10650, 2020.

Generative adversarial network (GANs)

Assume $\mathbf{x}, \mathbf{z}$ are random variables driven by the model:

$$\mathbf{z} \sim p(\mathbf{z}) = \mathcal{N}(\mathbf{0}_M, I_M) \quad (\text{latent noise distribution})$$

$$\mathbf{x} = G_\phi(\mathbf{z}) \quad (\text{generator})$$

where

- $\mathbf{z} \in \mathcal{Z}^M$ is the **continuous latent** variable,
- $\mathbf{x} \in \mathcal{X}$ is the **observed** variable,
- the function $G_\phi: \mathcal{Z}^M \rightarrow \mathcal{H}$ **neural net** called the **generator**

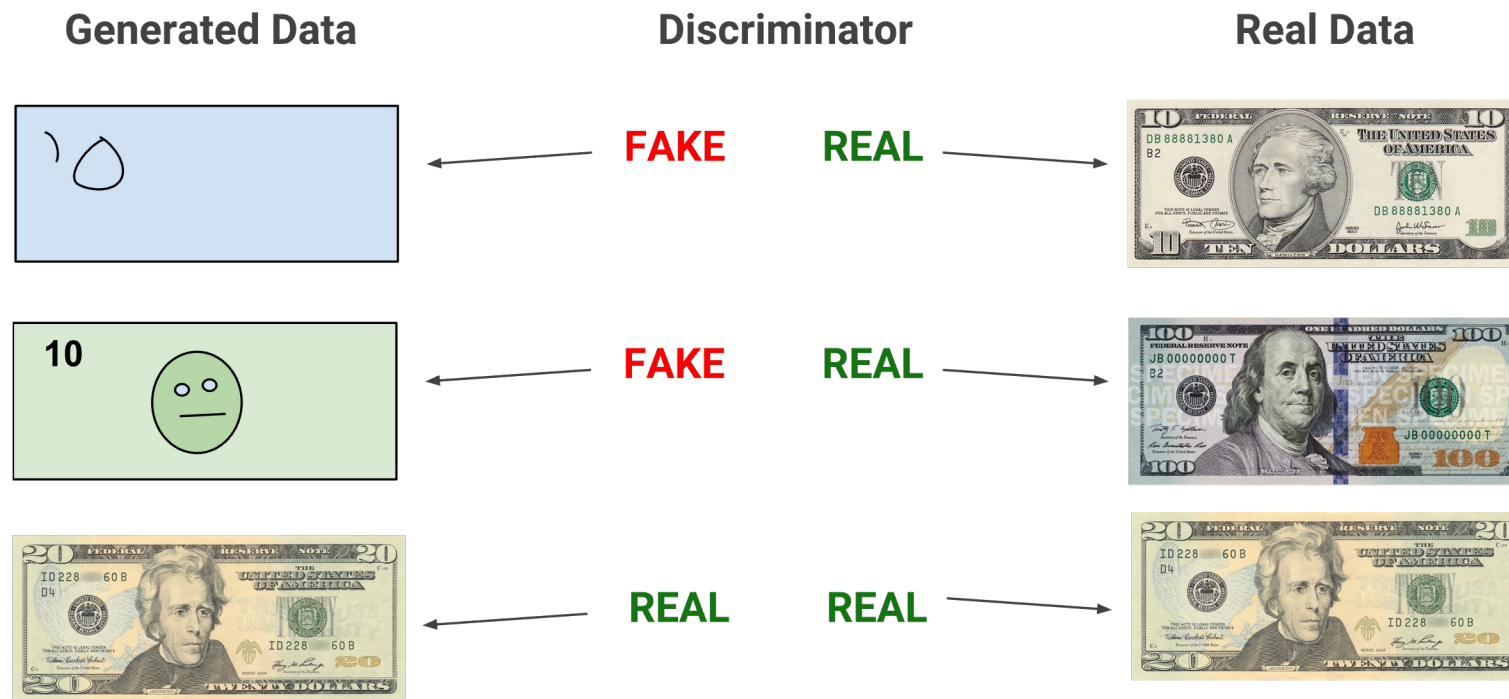
Advantage: got rid of output density

Issue: $p(\mathbf{x})$ only well-defined (via change-of-variables) if g_ϕ is invertible

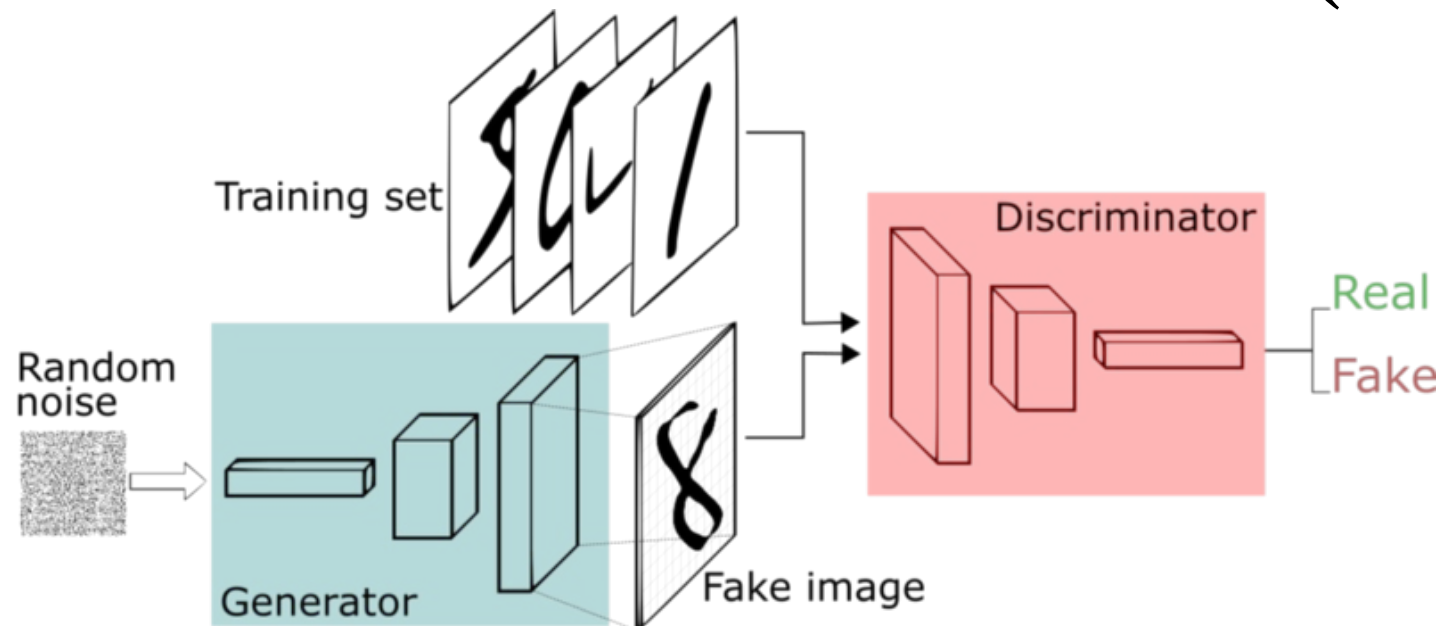
Why is this an issue?

Generative adversarial network (GANs)

- A GAN consists of:
 - A **generator** that learns to generate plausible data
 - A **discriminator** that learns to distinguish between fake (generated) and real data



Generative adversarial network (GANs)



The model is learned through a minimax game

$$\min_G \max_D \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} [1 - D(G(\mathbf{z}))]$$

- The **discriminator** $D(x)$ returns the probability that x is **real**
- The **generator** $G(z)$ transforms noise z to an image x .

GAN examples



2014



2015



2016



2017

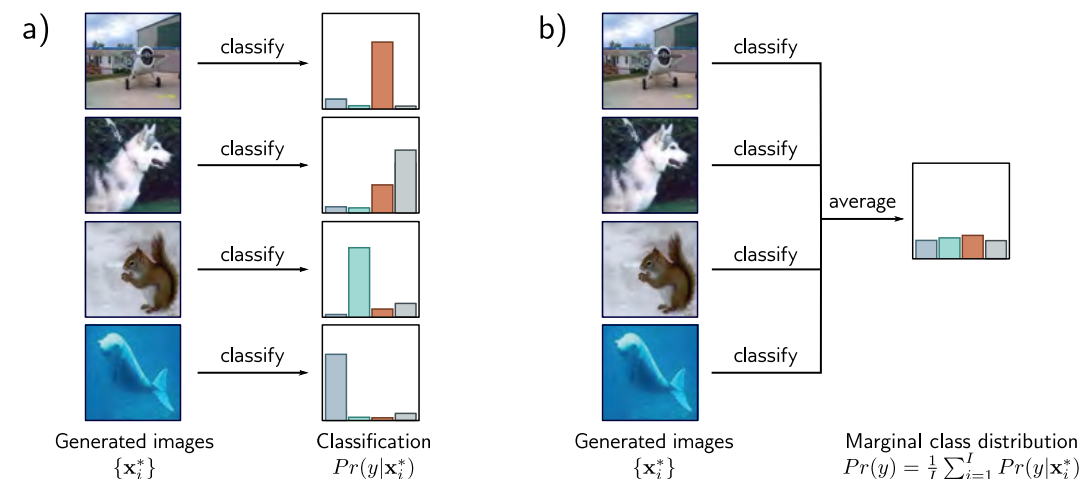
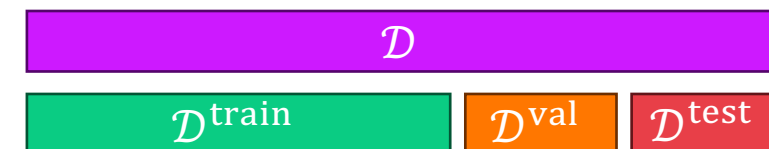
Measuring performance

- We can compare **test-set log-likelihoods**
- What about GANs?

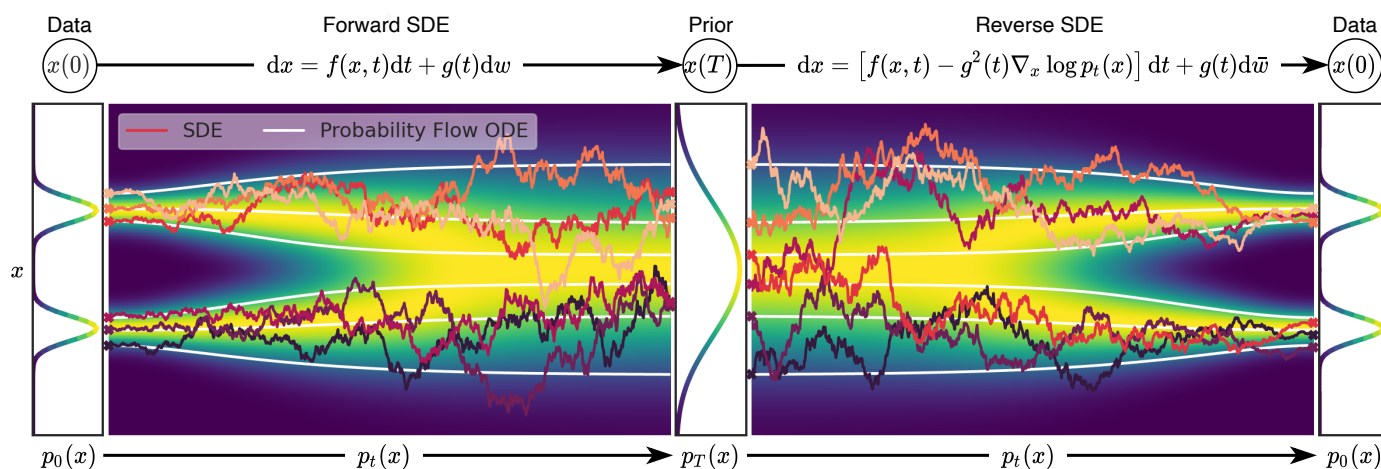
- **Inception score**: average distance between generated images class and test-set image classes

$$IS = \exp\left(\mathbb{E}_{\mathbf{x} \sim p_{\text{test}}(\mathbf{x})} [\text{KL}[p(y|\mathbf{x}) || p_{\text{test}}(y)]]\right)$$

- **Fréchet inception distance**: Wasserstein-2 distance between two Gaussians fitted to the last pooling-layer an Inception-v3 network.



SOTA: Diffusion models and variants



Want to learn more about this?
02460 Advanced Machine Learning
(Springs)

Today's exercises!

- Three Python notebook:
 - **Notebook 7.1:** Autoencoders
 - **Notebook 7.2:** VAEs
 - **Notebook 7.3:** GANs
- Two **problems** from the book on covered material

This was the last lecture!

Thank you!